ASSIGNMENT – 9.5

2303A51091
Batch : 29

Problem 1: String Utilities Function
Consider the following Python function:
def reverse_string(text):
return text[::-1]
Task:
1. Write documentation in:
o (a) Docstring
o (b) Inline comments
o (c) Google-style documentation
2. Compare the three documentation styles.
3. Recommend the most suitable style for a utility-based string library.

task1.py > ...

```python
def reverse_string(text):
    """Reverses the input string.

    Args:
        text (str): The string to reverse.

    Returns:
        str: The reversed string.
    """
    return text[::-1]


# ---- User Input ----
user_text = input("Enter a string to reverse: ")
result = reverse_string(user_text)
print("Reversed String:", result)
```

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS                    Python

/usr/bin/python3 "/Users/arepallyramcharan/ai assistent.py/assignment 9.5.py/task1.py"
arepallyramcharan@Arepallys-MacBook-Air assignment 9.5.py % /usr/bin/python3 "/Users/arepallyramcharan/ai assist
ent.py/assignment 9.5.py/task1.py"
Enter a string to reverse: cha6an
Reversed String: na6ahc
arepallyramcharan@Arepallys-MacBook-Air assignment 9.5.py %
```

Observation:

1. Docstrings provide a clear description of the function and its purpose.
2. Inline comments explain the logic step-by-step inside the code.
3. Google-style documentation offers a structured and standardized format.
4. Inline comments are useful for beginners but not suitable for automated documentation.
5. Google-style docstrings are best for utility libraries because they support documentation tools.

Problem 2: Password Strength Checker
Consider the function:
def check_strength(password):
return len(password) >= 8
Task:
1. Document the function using docstring, inline comments, and Google style.
2. Compare documentation styles for security-related code.
3. Recommend the most appropriate style.

task2.py > ...

```python
def check_strength(password):
    """Checks if password length is at least 8 characters.

    Args:
        password (str): Password entered by user.

    Returns:
        bool: True if strong, False otherwise.
    """
    return len(password) >= 8


# ---- User Input ----
user_password = input("Enter your password: ")

if check_strength(user_password):
    print("Password is strong.")
else:
    print("Password is weak. Must be at least 8 characters.")
```

Output:

```
arepallyramcharan@Arepallys-MacBook-Air assignment 9.5.py % /usr/bin/python3 "/Users/arepallyramcharan/ai assist
ent.py/assignment 9.5.py/task2.py"
Enter your password: armdgufiie
Password is strong.
arepallyramcharan@Arepallys-MacBook-Air assignment 9.5.py %
```

Observation:

1. Security-related functions require clear and structured documentation.
2. Docstrings help explain the purpose and return values of the function.
3. Inline comments clarify the validation logic used in the function.
4. Google-style documentation improves readability and maintainability.
5. For security modules, Google-style documentation is recommended due to its clarity and professional standard.

Problem 3: Math Utilities Module
Task:
1. Create a module math_utils.py with functions:
o square(n)
o cube(n)
o factorial(n)
2. Generate docstrings automatically using AI tools.
3. Export documentation as an HTML file.

task3.py > ...

```python
def square(n):
    return n * n


def cube(n):
    return n * n * n


def factorial(n):
    if n < 0:
        raise ValueError("Factorial not defined for negative numbers")

    if n == 0 or n == 1:
        return 1

    result = 1
    for i in range(2, n + 1):
        result *= i
    return result


# ---- User Input ----
num = int(input("Enter a number: "))

print("Square:", square(num))
print("Cube:", cube(num))
print("Factorial:", factorial(num))
```

Output:

```
arepallyramcharan@Arepallys-MacBook-Air assignment 9.5.py % /usr/bin/python3 "/Users/arepallyramcharan/ai assist
ent.py/assignment 9.5.py/task3.py"
Enter a number: 34
Square: 1156
Cube: 39304
Factorial: 295232799039604140847618609643520000000
arepallyramcharan@Arepallys-MacBook-Air assignment 9.5.py %
```

Ln 27, Col 36 Spaces: 4 UTF-8

Observation:

- Docstrings allow automatic documentation generation using tools like pydoc.
- Proper documentation helps developers understand mathematical functions easily.
- HTML documentation makes modules accessible through browsers.
- Structured documentation improves the maintainability of utility modules.
- Using standardized documentation practices ensures consistency across the project.

Problem 4: Attendance Management Module

Task:

1. Create a module attendance.py with functions:

o mark_present(student)

o mark_absent(student)

o get_attendance(student)

2. Add proper docstrings.

3. Generate and view documentation in terminal and browse

task4.py > ...

```python
attendance_record = {}

def mark_present(student):
    attendance_record[student] = "Present"

def mark_absent(student):
    attendance_record[student] = "Absent"

def get_attendance(student):
    return attendance_record.get(student, "Not Recorded")


# ---- User Input ----
while True:
    print("\n1. Mark Present")
    print("2. Mark Absent")
    print("3. Get Attendance")
    print("4. Exit")

    choice = input("Enter your choice: ")

    if choice == "1":
        name = input("Enter student name: ")
        mark_present(name)
        print("Marked Present.")

    elif choice == "2":
        name = input("Enter student name: ")
        mark_absent(name)
        print("Marked Absent.")

    elif choice == "3":
        name = input("Enter student name: ")
        print("Attendance Status:", get_attendance(name))

```

Output:

```
arepallyramcharan@Arepallys-MacBook-Air assignment 9.5.py % /usr/bin/python3 "/Users/arepallyramcharan/ai assist
ent.py/assignment 9.5.py/task4.py"

1. Mark Present
2. Mark Absent
3. Get Attendance
4. Exit
Enter your choice: 1
Enter student name: ram
Marked Present.
```

Observation:

1. Documentation helps clarify the purpose of each attendance function.
2. Docstrings allow tools to automatically generate module documentation.
3. Properly documented modules are easier for teams to maintain.
4. Terminal and browser documentation improves accessibility for developers.
5. Clear documentation ensures accurate usage of attendance functions.

Problem 5: File Handling Function

Consider the function:

def read_file(filename):

with open(filename, 'r') as f:

return f.read()

Task:

1. Write documentation using all three formats.

2. Identify which style best explains exception handling.

3. Justify your recommendation.

task5.py > ...

```python
def read_file(filename):
    """Reads contents of a file."""
    with open(filename, 'r') as f:
        return f.read()


# ---- User Input ----
file_name = input("Enter file name to read: ")

try:
    content = read_file(file_name)
    print("\nFile Content:\n")
    print(content)

except FileNotFoundError:
    print("Error: File not found.")

except PermissionError:
    print("Error: Permission denied.")

except Exception as e:
    print("Unexpected error:", e)
```

Output:

```
arepallyramcharan@Arepallys-MacBook-Air assignment 9.5.py % /usr/bin/python3 "/Users/arepallyramcharan/ai assist
ent.py/assignment 9.5.py/task5.py"
Enter file name to read: ram
Error: File not found.
arepallyramcharan@Arepallys-MacBook-Air assignment 9.5.py %
```

Ln 22, Col 34 Spaces: 4 UTF-8

Observation:

1. File handling functions require documentation for parameters and returned data.
2. Exception handling must be clearly described in the documentation.
3. Inline comments help explain file operations step-by-step.
4. Docstrings allow documentation tools to display function details automatically.
5. Google-style documentation is best for explaining exceptions and file operations clearly.